Examination Portal

Application Flow Documentation

Tech Stack: Spring Boot (Java) · React (Vite) · MySQL · JWT · BCrypt · Hibernate/JPA

Architecture: Single-page frontend (port 5173) → REST API backend (port 8080) → MySQL

1. Database Schema

1.1 Table Overview

Table	Purpose	Key Columns
users	Stores both admin and student accounts	id, username, email, password (BCrypt), role (ADMIN|STUDENT), is_active
subjects	Subject categories for tests	id, name, description, is_active
tests	Test/exam definitions — owned by an admin	id, title, subject_id, duration_minutes, total_marks, passing_marks, created_by
questions	Questions owned exclusively by one test	id, test_id (FK→tests), subject_id (nullable), question_text, option1-4, correct_option, marks (DECIMAL), difficulty
exam_attempts	One row per student exam session	id, student_id, test_id, start_time, end_time, score, status (IN_PROGRESS|COMPLETED|ABANDONED|EXPIRED)
student_answers	One row per question answered in an attempt	id, exam_attempt_id, question_id, selected_option, is_correct

1.2 Key Relationships

Relationship	Type	Cascade
tests → questions	OneToMany (test_id FK on questions)	ALL + orphanRemoval — deleting a test deletes all its questions
tests → subjects	ManyToOne	None
tests → users (created_by)	ManyToOne	None
exam_attempts → users	ManyToOne	CASCADE DELETE
exam_attempts → tests	ManyToOne	CASCADE DELETE
student_answers → exam_attempts	ManyToOne	CASCADE DELETE

2. Authentication Flow

2.1 Registration

Step	Actor	Action / API Call	DB Operation
1	User	Fill register form (username, email, password, role, name)	—
2	Frontend	POST /auth/register	—
3	AuthService	Check username/email uniqueness; validate role	SELECT users WHERE username=? or email=?
4	AuthService	BCrypt-encode password; save user with is_active=true	INSERT INTO users
5	JwtUtil	generateToken(username, role) — HMAC-SHA256, 24h expiry	—
6	Frontend	Store token in localStorage; redirect to dashboard	—

2.2 Login (supports username OR email)

Step	Actor	Action / API Call	DB Operation
1	User	Enter username or email + password	—
2	Frontend	POST /auth/login { username: "", password }	—
3	AuthService	findByUsername().or(findByEmail()) to resolve actual user	SELECT users WHERE username=? UNION email=?
4	AuthService	authenticationManager.authenticate(username, rawPassword)	Loads user via CustomUserDetailsService
5	AuthService	generateToken(username, role)	—
6	Frontend	Store token+user in localStorage + AuthContext; redirect to dashboard	—

2.3 JWT Token Details

Field	Value / Description
Algorithm	HMAC-SHA256 (HS256)
Claims	sub (username), role (ADMIN|STUDENT), iat (issued at), exp (expiry)
Expiry	86400000 ms = 24 hours
Storage	localStorage on the browser
Transmission	Authorization: Bearer header on every API request (Axios interceptor)

2.4 Request Authorization Pipeline

- 1. Browser sends request with Authorization: Bearer
- 2. JwtRequestFilter extracts and validates token — signature + expiry
- 3. CustomUserDetailsService loads user; checks is_active
- 4. SecurityContextHolder is populated with authenticated user
- 5. SecurityConfig URL rules: /admin/** requires ROLE_ADMIN, /student/** requires ROLE_STUDENT

- 6. @PreAuthorize("hasRole('ADMIN')") on AdminController provides method-level enforcement

3. Admin Flow

3.1 Subject Management

Operation	API Endpoint	DB Operation
List all subjects	GET /admin/subjects	SELECT * FROM subjects
Create subject	POST /admin/subjects { name, description }	INSERT INTO subjects
Update subject	PUT /admin/subjects/{id}	UPDATE subjects SET ...
Delete subject	DELETE /admin/subjects/{id}	DELETE FROM subjects WHERE id=?

3.2 Test Creation with Inline Questions

Questions are **not a global entity**. They are created inline during test creation and belong exclusively to that test. There is no separate Question Management page.

Step	Action	Detail
1	Admin fills test metadata	Title, description, subject, duration, total marks, passing marks
2	Admin adds questions inline	Each question card: question text, option 1–4, correct option, difficulty
3	Marks auto-distributed	marksPerQuestion = totalMarks ÷ questionCount — computed live in UI, applied on submit
4	POST /admin/tests	Payload: CreateTestRequest { title, description, subjectId, durationMinutes, totalMarks, passingMarks, questions: [QuestionDTO] }
5	TestService.createTest()	Creates Test, iterates QuestionDTO list, sets test+subject on each Question, saves all in one transaction
6	DB writes	INSERT INTO tests; INSERT INTO questions (test_id=...) for each question

3.3 Test Management API

Operation	API Endpoint	Notes
List all tests	GET /admin/tests	Returns tests with questionCount field
Get test by ID	GET /admin/tests/{id}	
Create test + questions	POST /admin/tests	Body: CreateTestRequest with inline questions
Update test metadata	PUT /admin/tests/{id}	Updates title, marks, duration etc.
Delete test	DELETE /admin/tests/{id}	Cascade deletes all owned questions automatically
Add question to existing test	POST /admin/tests/{testId}/questions	Body: QuestionDTO
Remove question from test	DELETE /admin/tests/{testId}/questions/{questionId}	

3.4 Admin Dashboard Stats

The admin dashboard fetches stats from 3 endpoints in parallel: **GET /admin/users**, **GET /admin/subjects**, **GET /admin/tests**. Total Questions is derived client-side by summing *questionCount* across all tests (no separate /admin/questions endpoint exists).

3.5 User Management

Operation	API Endpoint	DB Operation
List all users	GET /admin/users	SELECT * FROM users
Get by role	GET /admin/users/role/{role}	SELECT * FROM users WHERE role=?
Update user	PUT /admin/users/{id}	UPDATE users SET ...
Delete user	DELETE /admin/users/{id}	DELETE FROM users WHERE id=?

3.6 Reports

Report	API Endpoint	Returns
All students report	GET /admin/reports/students	List of students with attempt counts and avg scores
Subject-wise report	GET /admin/reports/subjects	Per-subject stats: total tests, attempts, avg score

4. Student Flow

4.1 Browse Available Tests

When the student opens the Available Tests page, two API calls are made in parallel:

API Call	Purpose	Used For
GET /student/tests	Fetch all active tests	Render test cards with title, subject, duration, marks, question count
GET /student/exams/attempts	Fetch student's completed attempts	Build a set of completed test IDs to control card state

Test card states:

- Not attempted — badge shows "Active" (green), blue "Start Test" button enabled
- Already attempted — badge shows "Completed" (gray), "Already Attempted" label replaces button (no re-attempt allowed)
- Question count shown as test.questionCount (server-computed field, not from the questions array)

4.2 Start Exam

Step	API Call	DB Operation
Start exam	POST /student/exams/start/{testId}	INSERT INTO exam_attempts (student_id, test_id, start_time, status=IN_PROGRESS)
Load questions	GET /student/exams/{attemptId}/questions	Returns questions for the test — correct_option is withheld from response

4.3 Answering & Submitting

Step	API Call	DB Operation / Logic
Submit answer	POST /student/exams/answer?attemptId=X { questionId, selectedOption }	UPSERT student_answers; is_correct = (selected == correct_option)
Submit exam	POST /student/exams/submit { attemptId, answers[] }	Saves all answers; calculates score = correct × (totalMarks / questionCount); UPDATE exam_attempts status=COMPLETED

4.4 Results & Performance

Feature	API Call	Returns
View attempt result	GET /student/exams/attempts/{attemptId}	Score, pass/fail, correct/wrong/unanswered counts
Full answer review	GET /student/exams/{attemptId}/answers	Each question with selected_option, correct_option, is_correct
Performance history	GET /student/performance	All past attempts with scores and test names
Subject-wise performanc e	GET /student/performance/subjects	Per-subject avg, min, max scores
Email results	POST /student/email-results/{attemptId}	Sends exam result email to student's registered address

5. Frontend Architecture

5.1 Key Files

File / Folder	Purpose
src/context/AuthContext.jsx	Global auth state — user, token, login(), logout(), register()
src/services/api.js	Axios instance — baseURL :8080, interceptor auto-attaches Bearer token, redirects on 401
src/App.jsx	Router — public routes, protected admin routes (/admin/**), protected student routes (/student/**)
src/components/Layout.jsx	Shared nav bar — admin links: Dashboard, Users, Subjects, Tests, Reports, Profile
src/pages/admin/TestManagement.jsx	Logic — inlineQuestions state, marksPerQuestion computed live, handleSubmit sends CreateTestRequest
src/components/admin/TestManagementComponent.jsx	UI — test form + inline question builder cards, read-only marks display, live marks banner
src/pages/student/AvailableTests.jsx	Logic — fetches tests + completed attempts in parallel; derives completedTestIds set
src/components/student/AvailableTestsComponent.jsx	UI — test cards; shows Completed badge and disables button for already-attempted tests; uses test.questionCount
src/pages/student/ExamPage.jsx	Exam timer, question navigation, answer submission
src/pages/student/TestReport.jsx	Post-exam review showing correct/wrong answers per question

5.2 Admin Navigation

- Dashboard — overview stats (users, subjects, tests, total questions)
- Users — manage admin/student accounts
- Subjects — create and manage subjects
- Tests — create tests with inline questions
- Reports — student performance and subject-wise analytics
- Profile — admin account details

6. Marks Distribution Logic

Marks are **auto-distributed equally** across all questions at test creation time.

Stage	Where	Logic
Live preview	Frontend (TestManagementComponent)	marksPerQuestion = totalMarks ÷ questionCount — updates as questions are added/removed; shown in blue banner
On submit	Frontend (TestManagement.handleSubmit)	Each QuestionDTO.marks = parseFloat((totalMarks / count).toFixed(2))
Stored	DB (questions.marks)	DECIMAL(5,2) — e.g. 10 marks ÷ 4 questions = 2.50 per question
Scoring	Backend (ExamService)	score = correctAnswers × (test.totalMarks / test.questionCount)